

CYBERCONFIG AI

Project Explanation & Continuation Guide

For a beginner teammate — what the project is, what is already built, how it works, and what we should build next.

1. What is our project?

CyberConfig AI is an **AI-powered network security compliance auditor**.

In simple words: we give the software a network device configuration, and it checks whether the configuration is secure or contains security problems.

2. What problem are we solving?

Companies use network devices from different vendors such as Cisco, Juniper, Fortinet, Palo Alto, and MikroTik. Different vendors can use different configuration syntax.

Manually checking many configurations takes time, security problems can be missed, and fixing configurations can be risky. Security teams also need evidence and reports for audits.

Our solution: Understand the configuration → Find the problem → Explain the problem → Suggest a fix → Verify the fix → Generate evidence.

3. What have we built so far?

We already have a **working MVP/prototype interface**. The current prototype contains: AuditorFleetNLP ParserCopilotEvidenceSIH Deck The prototype also demonstrates vendor presets, configuration auditing, risk findings, remediation recommendations, verification flow, and evidence/report functionality.

4. Vendor Selection

The prototype includes vendor presets such as Cisco Catalyst, Juniper SRX345, Fortinet FortiGate, Palo Alto, MikroTik, and Novel/Unknown.

The important idea is: **Vendor-specific syntax** → **Common security meaning**.

5. Configuration Ingestion

The user provides a network configuration file or configuration text. Examples include router, switch, and firewall configurations.

Current prototype: configuration upload/audit workflow.

Future: support more file types, API-based collection, and approved read-only SSH collection.

6. AI/NLP Parser

The parser reads raw configuration and tries to understand its security meaning.

Example:

```
set telnet disabled=no port=23
```

The system should understand: **Telnet is enabled**, and that it is an insecure remote-management protocol.

Flow: **Raw Configuration** → **Parser** → **AI/NLP Understanding** → **Security Meaning**.

7. Normalization

Different vendors may use different commands for the same security concept. The normalization layer converts them into a common internal security model so the same policy logic can work across vendors.

Example common representation:

Remote Management / Protocol: Telnet / Enabled: Yes / Port: 23

8. Policy Engine

The Policy Engine checks the normalized configuration against security/compliance rules such as CIS, NIST, and DISA STIG.

Example rule: IF Telnet = Enabled → Security Finding = High Risk

Core principle: AI proposes; Policy Engine validates.

9. Finding Detection

A finding means something is wrong or risky in the configuration.

Example: **Telnet Enabled**.

The system should show evidence from the configuration and explain why the finding matters.

10. Risk Classification

Findings can be grouped into **Critical, High, Medium, and Low**. The purpose is to help the security team decide which problems should be fixed first.

11. Explainable AI

The system should explain:

What? Telnet is enabled.

Evidence? The configuration contains the Telnet setting.

Why dangerous? It can expose sensitive communication and credentials.

Impact? Potential credential interception or other security exposure.

Recommendation? Disable Telnet and use secure remote management such as SSH.

12. Remediation

After detecting a problem, CyberConfig AI recommends a solution. Example: **Disable Telnet and enable secure SSH**.

Where supported, the system can provide a vendor-specific remediation command. We should not blindly apply AI-generated commands.

13. Human Approval

The administrator remains in control:

AI Recommendation → Proposed Change → Human Reviews → Approve → Apply

This is important because network configuration changes can affect live systems.

14. Re-scan & Verification

After applying an approved fix, the system checks the configuration again.

Before: Telnet = Enabled ■

After: Telnet = Disabled ■

This creates the closed-loop workflow: **Detect** → **Recommend** → **Approve** → **Apply** → **Re-scan** → **Verify**.

15. Evidence & Reporting

The Evidence section is designed to record audit information such as device, vendor, finding, risk level, evidence, compliance mapping, recommendation, and verification result. The prototype also demonstrates evidence/report generation and JSON export.

16. Security Copilot

The AI Security Copilot is intended to let a security administrator ask questions in simple language, for example: "Why is this device high risk?" or "What should I fix first?"

17. Dashboard

The dashboard is the main screen for the security administrator/CISO. It should quickly show total devices, compliant/non-compliant devices, Critical/High/Medium/Low risks, and top security problems.

Goal: understand the security situation within a few seconds.

18. My Main Role — Frontend + UI/UX

My major role is **Frontend Development and UI/UX**. The goal is to make a complex security system easy to use and understand.

The frontend connects backend results to the user interface:

Backend Finding → **User-Friendly Interface** → **Human Decision**.

19. What we should build next

Now we should move from a prototype/demo toward a connected software product.

Phase 1 — Backend: FastAPI, configuration upload, parser, policy engine, risk engine, database.

Phase 2 — Database: SQLite for early development; PostgreSQL for a more scalable deployment.

Phase 3 — Frontend ↔ Backend: upload configuration → API → processing → findings → dashboard.

The most important next step is making the frontend, backend, parser/policy logic, and database work together.

20. Final Target Architecture

USER → React Frontend → FastAPI API → Configuration Ingestion → AI/NLP Parser → Normalization → Policy Engine → Risk Engine → Findings → AI Remediation Advice → Human Approval → Safe Remediation → Re-scan & Verify → Evidence & Reports → Database

21. Real Example

Suppose a MikroTik router contains:

```
set telnet disabled=no port=23
```

CyberConfig AI should: read the configuration → understand Telnet is enabled → check the policy → generate a finding → assign risk → explain the impact → recommend disabling Telnet → wait for human approval → apply the approved change → re-scan → verify → generate evidence.

22. What a new teammate should learn first

Do not try to learn everything at once. Start with these topics:

1. What is a network configuration?
2. What is a security misconfiguration?
3. What is an AI/NLP parser?
4. What is a policy engine?
5. What is risk assessment?
6. How does FastAPI communicate with React?
7. How does the complete CyberConfig AI workflow work?

“CyberConfig AI reads network configurations, understands their security meaning, checks them against security policies, identifies and explains risks, recommends safe fixes, and verifies the result.”

Remember: AI proposes → Policy Engine validates → Human approves → System verifies.